

```
astype('float32')
# Set model input
module.set_input(x=input_data)

# Execute the Model
module.run()

# Get the first output
out = module.get_output(0)

# out is an NDArray type and out.asnumpy() can
# return an numpy array for the same.
```

The example given above explains the import, compilation and execution on target for a TensorFlow model on a x86 target using a Python runtime interface. Given below are a few choices for the usage of other options for frontends, hardware targets and programming languages.

We can compile various different target devices by changing `target='llvm'` in Step 3 above. TVM supports a wide range of targets like 'llvm', 'cuda', 'opencl', 'metal', 'rocm', 'vulkan', 'nvptx', 'llvm-device=arm_cpu', 'opencl-device=mali' and 'aocl_sw_emu'. We may also need to specify `target_host` while using accelerators from Linux hosts like CUDA, OpenCL, etc.

We may need to pass `target_host='llvm'`. Cross-compilation is also possible, bypassing additional options to LLVM. For example, we may pass `target_host='llvm -target=aarch64-linux-gnu'` for an Android platform.

Apart from Python, TVM supports various programming languages while deploying a compiled module. Please refer to <https://docs.tvm.ai/deploy/index.html> for other programming languages.

Cross-compilation and RPC

To be developer friendly, TVM supports RPC to cross-compile the model, and to deploy and test it. RPC enables faster development by remote-loading compiler output on target, and by setting input and getting the output from a target to the host seamlessly. If you are interested, refer to https://docs.tvm.ai/tutorials/cross_compilation_and_rpc.html#.

For Android developers, TVM provides an RPC application as a quick-start to compile a model, deploy it remotely via RPC and test it (https://github.com/dmlc/tvm/blob/master/apps/android_deploy/README.md#build-and-installation).

AutoTVM

Apart from the standard compilation process, TVM also provides a framework to infer the best hardware parameters customised for the real underlying hardware. The `tvm.autotvm` sub-module provides various APIs for this. Refer to <https://docs.tvm.ai/api/python/autotvm.html?highlight=autotvm#module-tvm.autotvm> to know more about AutoTVM.

Versatile Tensor Accelerator (VTA)

VTA is an open, generic and customisable deep learning accelerator with a complete TVM based compiler stack. It was designed to expose the most salient and common characteristics of mainstream deep learning accelerators. Together, TVM and VTA form an end-to-end hardware-software deep learning system stack that includes hardware design, drivers, a JIT runtime, and an optimising compiler stack based on TVM. Do refer to <https://docs.tvm.ai/vta/index.html> to know more about VTA.

Benchmark

Benchmark information for various models like densenet, mobilenet, resnet, squeezenet, etc, on different platforms like ARM CPU, ARM GPU, NVIDIA GPU and AMD GPU is available at <https://github.com/dmlc/tvm/wiki/Benchmark>. 

By: Siva Rama Krishna Reddy B.

The author takes keen interest in researching open source technologies like Android, artificial intelligence and deep learning. He can be reached at srk.it38@gmail.com.

www.efy.in

EFYGROUP
YOURS SINCE 1989

“
**Nothing except
the mint can make
money without
advertising.**
”



-Thomas
Babington Macaulay

